```
from hmmlearn.hmm import GMMHMM
import yfinance as yf
import pandas as pd
import numpy as np
import statsmodels.api as sm
from scipy.stats import chi2
from copulas.bivariate import Clayton

# Extraction of data from Yahoo Finance
symbols = ["BWPUSD=X", "ZARUSD=X"]
stock_data = yf.download(symbols, start="2015-01-01", end="2024-11-08",
progress=False)

# Filtering and removal of missing values
stock_data_close = stock_data["Close"]
stock_data_close = stock_data_close.dropna()
print(stock_data_close)

# Fitting VECM model
VECM_mod = sm.tsa.VECM(stock_data_close, k_ar_diff=2, coint_rank=1)
VECM_fit = VECM_mod.fit()
print(VECM_fit.summary())
residuals = VECM_fit.resid
print(residuals)

# Fix the random seed for consistent results
np.random.seed(40)

# Number of components (states) for the HMM
n_components = 2

# Step 4: Define and fitting the Gaussian Mixture HMM
n_components = 2  # Number of hidden states
n_mix = 2         # Number of Gaussian components per state

# Initialize the GMMHMM model
model = GMMHMM(n_components=n_components, n_mix=n_mix, covariance_type="full",
n_iter=2000, random_state=42)
model.fit(residuals)


# Step 5: Displaying the model parameters
print("Means of each Gaussian component in each state:")
print(model.means_)         # Means of each Gaussian component in each state
print("\nCovariances of each Gaussian component in each state:")
print(model.covars_)        # Covariances of each Gaussian component in each
state
print("\nTransition matrix:")
print(model.transmat_)      # Transition matrix
print("\nInitial state probabilities:")
print(model.startprob_)     # Initial state probabilities

# Step 6: Predicting hidden states
hidden_states = model.predict(residuals)
```

```
print("\nPredicted hidden states:", hidden_states)

# Defining a function to calculate the log-likelihood for a given number of
regimes (components)
def calculate_log_likelihood(n_components, n_mix, residuals):
    model = GMMHMM(n_components=n_components, n_mix=n_mix,
covariance_type="full", n_iter=1000, random_state=42)
    model.fit(residuals)
    return model.score(residuals)  # Log-likelihood of the data under the model

# Calculating log-likelihood for different components
n_mix = 2  # Number of Gaussian components per state

# Log-likelihood for HMM with 1 component (state)
log_likelihood_1 = calculate_log_likelihood(n_components=1, n_mix=n_mix,
residuals=residuals)

# Log-likelihood for HMM with 2 components (states)
log_likelihood_2 = calculate_log_likelihood(n_components=2, n_mix=n_mix,
residuals=residuals)

# Perform the Likelihood Ratio Test (LRT)
lr_statistic = 2 * (log_likelihood_2 - log_likelihood_1)

# Degrees of freedom is the difference in the number of parameters between the
models
# Degrees of freedom for HMM with n_components is 2*n_components^2 +
n_components
df = (2 * 2 ** 2 + 2) - (2 * 1 ** 2 + 1)

# Computing the p-value from the chi-squared distribution
p_value = 1 - chi2.cdf(lr_statistic, df)
print(f"Likelihood Ratio Statistic: {lr_statistic}")
print(f"Degrees of Freedom: {df}")
print(f"P-Value: {p_value}")

# Interpreting the result
if p_value < 0.05:
    print(f"Since the p-value is {p_value}, we reject the null hypothesis and
conclude that the model with 2 components is a better fit.")
else:
    print(f"Since the p-value is {p_value}, we fail to reject the null
hypothesis and conclude that the model with 1 component is sufficient.")

# Step 4: Clayton copula analysis
# DataFrame for copula
residuals_pdf = pd.DataFrame(residuals, columns=["u", "v"])

# Transformation to uniform marginals
def transform_uni_marginals(residuals_pdf):
    rank_res = residuals_pdf.rank(method="average")
    uniform_marg = rank_res / (len(residuals_pdf) + 1)
    return uniform_marg
```

```
res_uniform = transform_uni_marginals(residuals_pdf)

#residuals_uniform to a numpy array
res_uniform_np = res_uniform.values

# Fit Clayton copula
clayton_c = Clayton()
clayton_c.fit(res_uniform_np)

# Clayton copula parameter
theta = clayton_c.theta
print(f"Fitted Clayton copula parameter (theta): {theta}")

# Kendall tau
kt = theta / (theta + 2)
print("Kendall_tau:", kt)

# Spearman's rho
sp = theta / (theta + 1)
print("Spearman:", sp)

# Lower tail dependence
lambda_L = (2 * theta) / (theta + 2)
print(f"Lower tail dependence (lambda_L): {lambda_L}")
```